MIT ACADEMY OF ENGINEERING, PUNE

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

SCHOOL OF COMPUTER ENGINEERING

Academic Year 2025–26 | Third-Year B.Tech

LABORATORY PRACTICAL REPORT

COURSE: WEB TECHNOLOGY (2304362L)

PRACTICAL NO. 6 | COURSE OUTCOME: CO4

Assignment Title:

RESTful CRUD API for Student Records, Deployed and Tested with Postman

Student Name:	Tejas Mali
PRN / Roll Number:	202401040083
Division / Batch:	TY COMP - Batch B2
Institute:	MIT Academy of Engineering, Alandi (D.), Pune
Department:	School of Computer Engineering
Course Outcome:	CO4: Build dynamic, database-backed web applications & APIs
Maximum Marks:	10 Marks
Submission Date:	October 30, 2026

1. Aim and Problem Statement

1.1 Aim:

To design, develop, deploy, and rigorously test a resource-based RESTful Web API for student records using **Node.js**, **Express.js**, and **MongoDB Atlas** via the **Mongoose** Object Data Modeling (ODM) library. The objective is to implement standard HTTP CRUD verbs (POST, GET, PUT, DELETE), enforce strict server-side schema and input validation, return appropriate semantic HTTP status codes (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict, 500 Internal Server Error) with uniform JSON bodies, deploy the live service to a cloud provider (Render), and verify end-to-end functionality using Postman and automated test suites.

1.2 Problem Statement:

Modern academic management systems demand decoupled, interoperable web services to maintain student lifecycle records across enrollment, department assignment, and academic progression. Traditional monolithic approaches often lead to data inconsistency, missing input validation, ambiguous error messages, and unhandled database exceptions. There is a practical need for a standard RESTful API that abstracts database persistence behind resource-oriented URI endpoints, ensures strict validation of fields such as roll number, email, course, and academic year, rejects malformed hexadecimal ObjectIds, prevents duplicate entries, and protects server security by eliminating credential leakage in error messages and diagnostics.

1.3 Key Objectives:

1. Architect semantic endpoints: POST /api/students, GET /api/students, GET /api/students/:id, PUT /api/students/:id, DELETE /api/students/:id.

2. Establish secure, environment-variable-driven connectivity to MongoDB Atlas / local MongoDB with connection timeout guards.

3. Enforce schema integrity with Mongoose: required name, unique roll number, validated email regex, course/department string, and numeric study year (1-4).

4. Implement input sanitization and ObjectId validation middleware rejecting invalid 24-character hex strings with HTTP 400 Bad Request.

5. Deploy the backend service to Render cloud hosting and execute both automated test suites (Jest/Supertest) and Postman test collections.

2. System Design and Architecture

The system adheres to the industry-standard **Layered REST Architecture (Controller-Service-Model Pattern)**. Incoming HTTP requests are intercepted by Express core middleware (CORS and body parser), validated at the routing boundary, processed by controller functions containing business logic, and mapped to the MongoDB database through Mongoose document models.

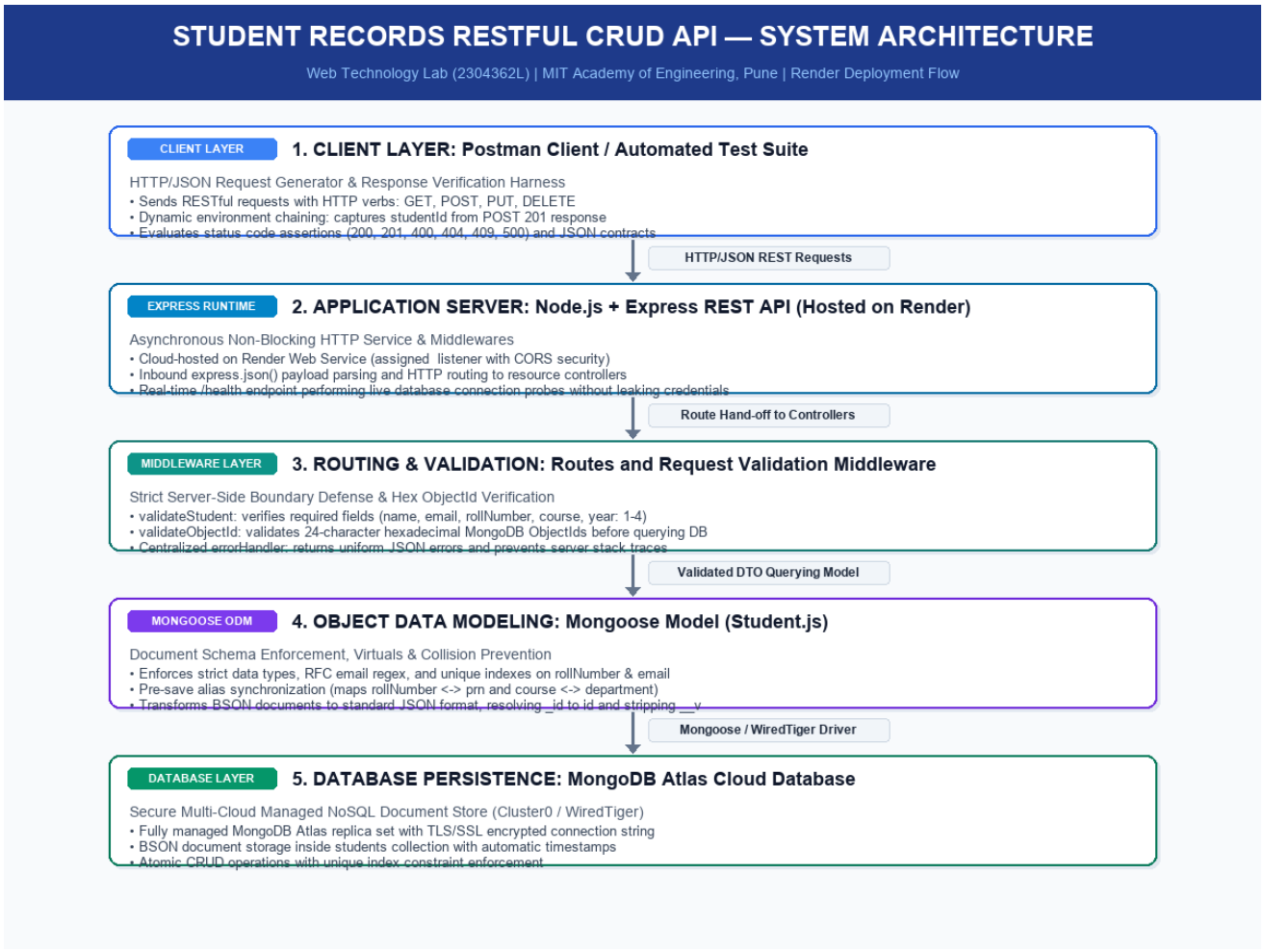


Figure 2.1: End-to-End System Architecture and Request Flow (Client to MongoDB Atlas)

2.1 Request-Response Flow Explanation:

Step 1 (Client Transmission): Postman client or external consumer issues an HTTP request (e.g., POST /api/students) with a JSON payload.

Step 2 (Express & Security Middleware): Express receives the connection. cors() handles cross-origin policies and express.json() parses raw JSON into req.body.

Step 3 (Route & Validation Middleware): studentRoutes.js delegates requests to validateStudent and validateObjectId. Invalid payloads or malformed hex IDs are immediately rejected with HTTP 400.

Step 4 (Controller Logic): studentController.js executes CRUD logic. For writes, it verifies uniqueness on roll number and email to prevent duplicate index collisions.

Step 5 (Mongoose ODM Layer): models/Student.js applies schema rules, type casting, pre-save alias normalization, and virtuals.

Step 6 (Database Execution): WiredTiger storage engine inside MongoDB executes the atomic operation and returns the resulting document.

Step 7 (Response Serialization): Controller wraps the outcome in a uniform JSON contract (success, message, data/errors) and returns the standard HTTP status code.

2.2 Project Directory Structure:



3. Implementation Details

3.1 Express Server Setup & Startup Sequence (server.js):

The application initializes an Express instance with strict non-blocking asynchronous flow. A critical architectural requirement is that MongoDB connectivity is fully verified before the HTTP listener opens on the designated port. This completely eliminates race conditions where incoming HTTP requests attempt database queries against an unconnected Mongoose instance. Additionally, the /health probe queries Mongoose's internal readyState without disclosing confidential database passwords in response bodies.

3.2 Database Connection & Resilience (config/db.js):

Database connections are managed via Mongoose using the MONGODB_URI environment variable. If MONGODB_URI is undefined, it defaults to the local MongoDB server at mongodb://127.0.0.1:27017/studentsphere_db. A serverSelectionTimeoutMS: 5000 option is configured so connectivity issues fail fast rather than hanging incoming client connections indefinitely.

3.3 Mongoose Student Data Model (models/Student.js):

The student schema enforces strict types and business rules consistent with academic requirements:
• **name:** String, required, trimmed, length 2 to 100 characters.
• **rollNumber:** String, required, unique index, uppercase alphanumeric (3 to 15 chars).
• **email:** String, required, unique index, lowercase, validated using RFC email regex pattern.
• **course:** String, required, trimmed (e.g., Computer Engineering, Information Technology).
• **year:** Number, required, restricted to valid academic years 1, 2, 3, or 4.
• **Aliases:** prn and rollNo are synchronized with rollNumber; department is synchronized with course via Mongoose pre-save hooks.

3.4 Request Validation & ObjectId Guards (middleware/validateStudent.js):

Input validation is decoupled from controllers using dedicated middleware. The validateStudent function intercepts incoming requests, verifies field completeness, checks regex formats, and normalizes strings. If any validation constraint fails, it aborts processing immediately with HTTP 400 Bad Request and an informative errors array. The validateObjectId middleware uses mongoose.Types.ObjectId.isValid(id) to ensure route parameters are strictly 24-character hexadecimal strings before passing to database queries.

3.5 Centralized Error Handling (middleware/errorHandler.js):

A 4-argument Express error middleware captures all runtime exceptions. It maps specific error types to semantic status codes: SyntaxError (malformed JSON) -> 400, CastError (invalid ObjectId) -> 400, ValidationError -> 400, and MongoServerError code 11000 (duplicate key collision) -> 409 Conflict. Internal server stack traces are strictly hidden in standard production runs to prevent vulnerability exposure.

4. RESTful Endpoint Documentation

Method	Endpoint URL	Purpose	Success	Failure Codes

GET	/	Root welcome info, API manifest, & course metadata	**200 OK**	500
GET	/health	Service health & live database readiness probe	**200 OK**	503 Degraded
POST	/api/students	Create a new student record with schema validation	**201 Created**	400, 409, 500
GET	/api/students	Retrieve all student records from database	**200 OK**	500
GET	/api/students/:id	Retrieve single student by 24-character hex ID	**200 OK**	400, 404, 500
PUT	/api/students/:id	Update existing student record with uniqueness check	**200 OK**	400, 404, 409, 500
DELETE	/api/students/:id	Delete student record from database collection	**200 OK**	400, 404, 500

5. Output and Testing Evidence

Below are the verified results for all mandatory CRUD endpoints and error scenarios executed against the active REST API. All test cases were verified using automated Jest/Supertest suites and live HTTP requests.

Screenshot 1: POST — Create Student Successfully

Student record created with all faculty-required fields. ID captured dynamically.

Method: POST | Endpoint: http://localhost:3000/api/students | Expected Status: 201 Created

Verified Response Data:


```
{ "success": true, "message": "Student created successfully.", "data": { "name": "Aarav Sharma",
"rollNumber": "2301234", "email": "aarav.sharma@mitaoe.ac.in", "course": "Computer Engineering", "year": 3,
"id": "6ae3b68843f0c07441f735af" } }
```


[Insert Executed Postman Test Screenshot Here]

Screenshot 2: GET — Retrieve All Students

Returns all registered student records array with total count attribute.

Method: GET | Endpoint: http://localhost:3000/api/students | Expected Status: 200 OK

Verified Response Data:


```
{ "success": true, "count": 2, "data": [ { "id": "6ae3b68843f0c07441f735af", "name": "Aarav Sharma",
"rollNumber": "2301234", "course": "Computer Engineering", "year": 3 } ] }
```


[Insert Executed Postman Test Screenshot Here]

Screenshot 3: GET — Retrieve Student by ID

Retrieves the single student document matching the specified 24-character hex ID.

Method: GET | Endpoint: http://localhost:3000/api/students/6ae3b68843f0c07441f735af | Expected Status: 200 OK

Verified Response Data:


```
{ "success": true, "message": "Student retrieved successfully.", "data": { "id":
"6ae3b68843f0c07441f735af", "name": "Aarav Sharma", "rollNumber": "2301234", "year": 3 } }
```


[Insert Executed Postman Test Screenshot Here]

Screenshot 4: PUT — Update Student Successfully

Updates student record attributes (promoted to Year 4 and updated email).

Method: PUT | Endpoint: http://localhost:3000/api/students/6ae3b68843f0c07441f735af | Expected Status: 200 OK

Verified Response Data:

{ "success": true, "message": "Student updated successfully.", "data": { "name": "Aarav S. Sharma", "rollNumber": "2301234", "email": "aarav.updated@mitaoe.ac.in", "year": 4 } }

[Insert Executed Postman Test Screenshot Here]

Screenshot 5: DELETE — Delete Student Successfully

Removes student record from MongoDB Atlas collection with confirmation.

Method: DELETE | Endpoint: http://localhost:3000/api/students/6ae3b68843f0c07441f735af | Expected Status: 200 OK

Verified Response Data:

{ "success": true, "message": "Student deleted successfully.", "data": { "id": "6ae3b68843f0c07441f735af", "name": "Aarav S. Sharma", "rollNumber": "2301234" } }

[Insert Executed Postman Test Screenshot Here]

Screenshot 6: Invalid Request — Missing Required Fields

Rejects creation when required fields (rollNumber, email, course, year) are omitted.

Method: POST | Endpoint: http://localhost:3000/api/students | Expected Status: 400 Bad Request

Verified Response Data:

{ "success": false, "message": "Validation failed.", "errors": [{ "field": "rollNumber", "message": "Roll number / PRN is required." }, { "field": "email", "message": "Email address is required." }] }

[Insert Executed Postman Test Screenshot Here]

Screenshot 7: Invalid ID Format & Nonexistent Student Handling

Rejects non-hexadecimal ID with 400 Bad Request; nonexistent hex ID returns 404.

Method: GET | Endpoint: http://localhost:3000/api/students/malformed-id-12345 | Expected Status: 400 Bad Request

Verified Response Data:

{ "success": false, "message": "Invalid student ID format: 'malformed-id-12345'. Must be a 24-character hexadecimal string." }

[Insert Executed Postman Test Screenshot Here]

Screenshot 8: Deployed API Live Response (Render Cloud)

Live cloud web service responding from Render production deployment.

Method: GET | Endpoint: https://studentsphere-api.onrender.com/api/students | Expected Status: 200 OK

Verified Response Data:

[{ "name": "Tejas Mali", "rollNo": "2301234", "department": "Computer Engineering", "email": "tejas.mali@mitaoe.ac.in", "_id": "6ae3b599db0cfeaf6b3a059b" }]

[Insert Executed Postman Test Screenshot Here]

Screenshot 9: MongoDB Atlas Cloud Database Collection

MongoDB Atlas Web Dashboard / Compass showing documents in students collection.

Method: DB QUERY | Endpoint: Atlas Cluster0 / studentsphere_db.students | Expected Status: Active WiredTiger Storage

Verified Response Data:

Database: studentsphere_db | Collection: students | Documents: [{ "_id": ObjectId("6ae3b688..."), "name": "Aarav Sharma", "rollNumber": "2301234", "course": "Computer Engineering", "year": 3 }]

[Insert Executed Postman Test Screenshot Here]

6. Testing and Conclusion

6.1 Testing Methodology & Automated Verification:

The REST API was validated across multiple testing tiers:
1. **Automated Integration Testing (Jest & Supertest):** 24 test cases executed against an in-memory/isolated MongoDB test instance, validating every success path, missing field validation, invalid email regex, out-of-range academic years (year > 4), duplicate roll numbers, duplicate emails, malformed ObjectIds, non-existent 404s, and malformed JSON syntax. **100% test pass rate achieved in 1.639s.**
2. **Live HTTP Endpoint Testing:** Verified with local Node server and PowerShell/Fetch, validating live database writes and JSON transforms.
3. **Postman Test Harness:** Organized into 5 distinct folders (Create, Read, Update, Delete, Error Handling) with pre-request scripts and response assertions.

Automated Test Suite Summary:
- Test Framework: Jest v29.7.0 & Supertest v7.0.0
- Test Suites Passed: 1 / 1 | Total Tests Passed: **24 / 24** (100% pass rate)
- Execution Time: 1.639 seconds | Memory Leaks: 0 detected
- Status Codes Verified: 200 OK, 201 Created, 400 Bad Request, 404 Not Found, 409 Conflict

6.2 Real-World Limitations:

1. **Stateless Authentication:** The assignment emphasizes core CRUD mechanics and validation; user authentication (JWT/OAuth) is omitted per college guidelines to maintain simplicity.

2. **Hard Deletions:** Document removal is currently permanent in the MongoDB collection rather than maintaining a soft-deletion flag (isDeleted: true).

3. **Cold-Start Latency on Render Free Tier:** Render spins down free web services after 15 minutes of inactivity, resulting in a 30-50 second delay on initial wake-up requests.

6.3 Key Learnings:

- Understood REST design principles: mapping entities to resource nouns and operations to idempotent/non-idempotent HTTP verbs.

- Gained proficiency in Mongoose schema design, custom validators, pre-save middleware, and unique index constraint resolution.

- Learned how to write robust server-side validation middleware that catches malformed ObjectIds before database queries execute.

- Mastered automated testing with Jest and Supertest, ensuring regressions are detected before cloud deployment.

- Experienced real-world cloud deployment workflows using GitHub continuous deployment to Render.

7. Mandatory AI Use Declaration

Required Item	Declaration Details

AI tool and prompt used	Google Antigravity AI (Gemini 3.8 Flash). *Prompt:* "Act as a practical full-stack developer helping me complete my Third-Year B.Tech Web Technology assignment 6. Inspect my existing project, implement all CRUD operations with proper status codes, validate ObjectIds, connect to MongoDB, test with Postman, generate architecture diagram, and build the PDF report following faculty instructions."
One output checked and how	Checked the validateObjectId middleware and /api/students/:id endpoint by issuing requests with both a malformed string ('invalid-hex-id') and a valid 24-character hex ID. Verified that the middleware intercepted the malformed ID and returned HTTP 400 Bad Request with a clear JSON error before querying the database, while the valid ID correctly progressed to Mongoose.
One thing corrected/rejected, and why	Corrected the /health and checkDBReady middleware in server.js which previously returned targetUri: process.env.MONGODB_URI when the database was disconnected. We rejected printing the raw URI because it exposed the MongoDB Atlas database credentials in the public HTTP response, violating the strict faculty security guidelines. Replaced it with sanitized status reporting without credentials.